

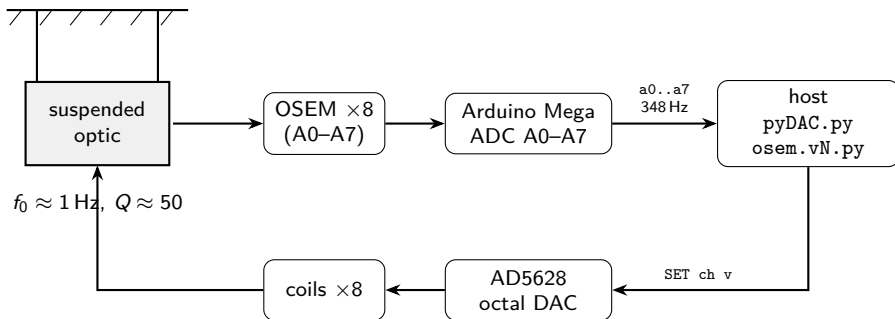
OSEMa bin-laden

Ten rungs of a suspended-optic damping loop: v0 to v10

6 August 2026

Four OSEM shadow sensors, four coils, one suspended mirror,
and eleven attempts at the supervisor around them.

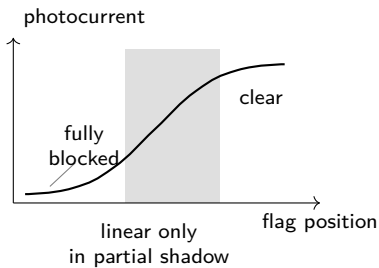
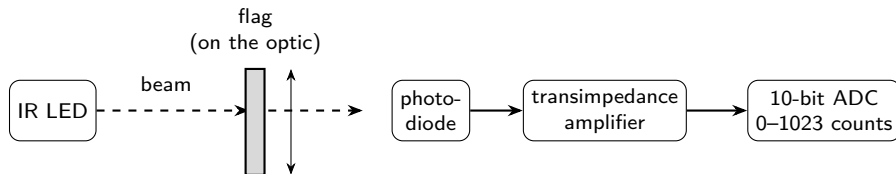
What is actually on the bench



Sensor index $i \rightarrow$ pin $Ai \rightarrow \text{DAC_CHANNELS}[i] = [1, 3, 5, 7]$ (rewired 2026-08-03).

All eight OSEMs and all eight coils are connected (scope, 2026-08-06). Every controller v0–v10 drives **A0–A3** — the reason is a measurement, and it is in Part IV.

The OSEM: a shadow sensor



- ▶ Sensing scale: $A_{VCC} = 5.02 \text{ V}$ over 1023 counts.
- ▶ Actuation scale: DAC full scale 2.5 V; the loop restricts itself to $V_{MIN}, V_{MAX} = 0.0, 0.5 \text{ V}$ around $BIAS = 0.25$.
- ▶ Pegged at either end the bandpass sees a flat top and the velocity estimate spikes — exactly when the loop is asked to push hardest.

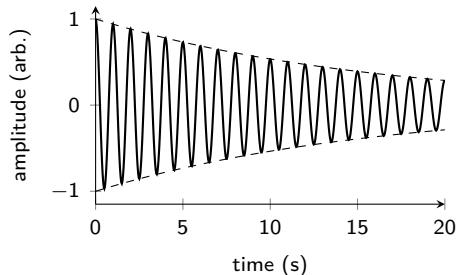
The pendulum, and why it needs help

$$\ddot{x} + \frac{\omega_0}{Q} \dot{x} + \omega_0^2 x = \frac{F(t)}{m}$$

$$\omega_0 = 2\pi f_0, \quad f_0 \approx 1 \text{ Hz}, \quad Q \approx 50$$

$$\tau = \frac{Q}{\pi f_0} \approx 16 \text{ s}$$

A suspension is built to be lossless. That is exactly what makes it ring: energy put in at 1 Hz stays in for tens of seconds.



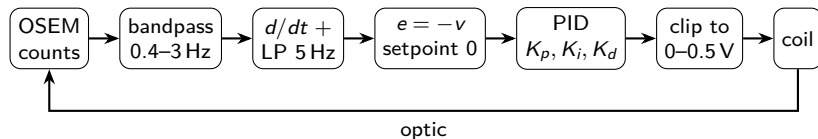
Illustrative: $e^{-t/\tau} \cos 2\pi f_0 t$ at $\tau = 16 \text{ s}$, $f_0 = 1 \text{ Hz}$. Not measured data.

Cold damping: the process variable is *velocity*

$$\text{bp}(t) = \text{LP}_{3\text{ Hz}} \left[\text{HP}_{0.4\text{ Hz}}[A(t)] \right]$$

$$v(t) = \text{LP}_{5\text{ Hz}} \left[\frac{\text{bp}(t) - \text{bp}(t - \Delta t)}{\Delta t} \right]$$

$$u(t) = -K(t) \cdot v(t)$$



The setpoint is a constant zero, so $e = -v$: the loop does not ask where the optic *is*, only that it stop *moving*. *provenance.md*, "The documented algorithm"

Three gains: P is a damper, I is a spring, D is a mass

With $e = -v$ and $v = \dot{x}$:

$$\underbrace{K_p e = -K_p \dot{x}}_{\text{force} \propto \text{velocity}}$$

$$\underbrace{K_i \int e dt = -K_i x}_{\text{force} \propto \text{displacement}}$$

$$\underbrace{K_d \frac{de}{dt} = -K_d \ddot{x}}_{\text{force} \propto \text{acceleration}}$$

term	acts on	physically	effect on the pole
K_p	velocity	viscous damper	the only term that removes energy
K_i	displacement	added spring	moves the resonance; does not damp
K_d	acceleration	added (negative) mass	noisiest term — differentiates twice

“Expect the amplitude ratio to end up no better than $v1$ ’s, because neither new term dissipates. What they buy is a stiffer, heavier effective plant.” *osem.v2.py docstring*

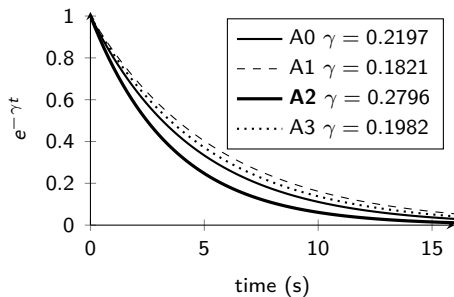
Where the three numbers came from

	ch0	ch1	ch2	ch3
STEADY_GAIN	-0.030	-0.030	+ 0.010	-0.030
CAPTURE_GAIN	-0.035	-0.035	+ 0.012	-0.035
KI_GAIN	-0.040	-0.040	+ 0.040	-0.040
KD_GAIN	-0.0015	-0.0015	+ 0.0015	-0.0015

- ▶ **-0.030 is the only gain ever validated against a scope** — ch0, 2026-07-15 run 3: “real damping at gain = -0.03, 6× reduction, no rail clipping.”
- ▶ CAPTURE_GAIN is stronger and used only at large amplitude. The source says plainly it is **not independently validated**.
- ▶ K_i and K_d share each channel's *sign*, because they share one loop — but not its *magnitude*: ch2 runs $|K_i| = 0.040$ like the rest while its $|K_p|$ is 0.010.
- ▶ $I_CLAMP_V = 0.15$ is “only a backstop”. The real anti-windup is back-calculation at TRACK_TC_S = 0.5 s.

Sources: comment block above STEADY_GAIN in *osem.v9.py*; *provenance.md* Table 1 and § “What the report does NOT contain”.

ch2 runs *positive* — on purpose



Envelopes drawn from the *measured* decay constants,
provenance.md Table 1 ($R^2 = 0.65\text{--}0.92$).

A2 damps **fastest** of the four — while running at $K = +0.010$, where A0/A1/A3 ran at -0.030 .

A positive gain on a normally-wired channel *pumps*. The only reading that fits is that **ch2's coil or OSEM is mounted the other way round**.

Confirmed on hardware 2026-08-04: ch2 ran positive alongside three negative channels and came out the best-damping axis, ratio 0.01–0.03 against ch0's 0.08.

This is not a bug and must never be “corrected”. The simulator encodes the same fact as `COIL_GAIN[2] = -4.15`.

$K_p = -0.040$: the least-evidenced number in the repo

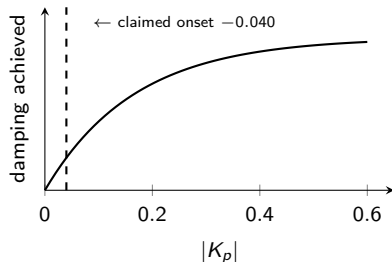
What is claimed. One sentence, in the docstring of `osem.v0.py` (deleted 2026-08-04, git history only):

real damping at gain = -0.03 ; onset of instability / rail at gain = -0.04 .

What supports it.

- ▶ The report, written two days later, never mentions it.
- ▶ No logged run in data/ records it.
- ▶ The simulator *provably cannot* reproduce it: the plant is linear apart from the ADC. Swept to -0.600 , damping increases monotonically and there are **zero faults**.
- ▶ v0 measured it with *one* channel driving; v1 then ran four at -0.030 — $4\times$ the total authority — without trouble.

Treat it as real and uncharacterised. Do not exceed -0.035 unattended.



Schematic — the simulator's qualitative trend, not measured data. *No* experiment behind the dashed line exists, and the ceiling actually used is -0.035 .

The gain schedule, and the baseline everything scales off

$$\text{RMS}_{\text{base}} = \sqrt{\frac{1}{N} \sum \text{bp}(t_i)^2} \quad \text{at gain} = 0$$

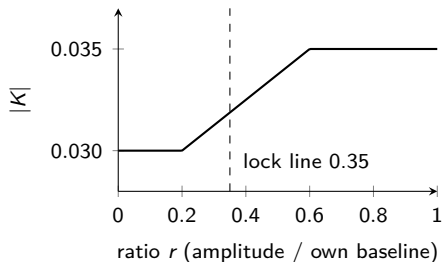
$$r(t) = \text{RMS}_{\text{win}}[\text{bp}(t)] / \text{RMS}_{\text{base}}$$

$$K(t) = K_{\text{steady}} + s(r) (K_{\text{cap}} - K_{\text{steady}})$$

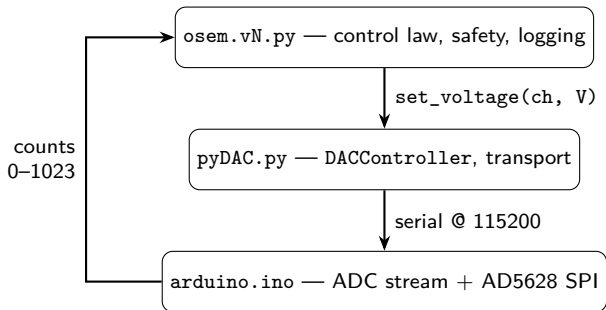
$$s(r) = \text{clip}\left(\frac{r-0.2}{0.4}, 0, 1\right)$$

Slew-limited at $\text{GAIN_SLEW_PER_S} = 0.02$, so there is no gain step that could itself kick a transient. That ramp doubles as the startup soft-start: $0 \rightarrow \text{capture gain}$ in 1.75 s.

One number, three jobs: the runaway breaker trips above $1.8\times$ baseline, the scheduler blends between $0.2\times$ and $0.6\times$, and **LOCKED** means **below $0.35\times$ held for 5 s.**



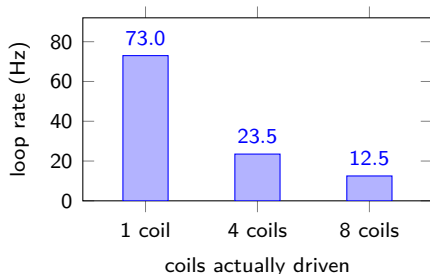
Three layers, one wire



direction	message	meaning
MCU → host	READY	sent once after setup()
host → MCU	SET <ch> <volts>	sets DAC 0-7; replies OK ch=.. v=..
host → MCU	STREAM / STOP	replies STREAMING / STOPPED
MCU → host	a0,...,a7	free-running ADC counts, one line per loop

Opening the port toggles DTR and resets the board, and setup() zeroes all eight DAC channels —
connecting is itself a safing action.

The wire is the bottleneck — and the loop rate falls with coil count



Median DAMPING-row spacing over 20 bench logs: 13.7

/ 42.6 / 79.9 ms. The wire itself delivers **348 Hz**.

analysis/out/loop_rate.csv

set_voltage blocks reading up to 50 stream lines waiting for its OK. **Every coil write costs a serial round trip**, so the loop rate is set by how many coils are driven, not by the control law.

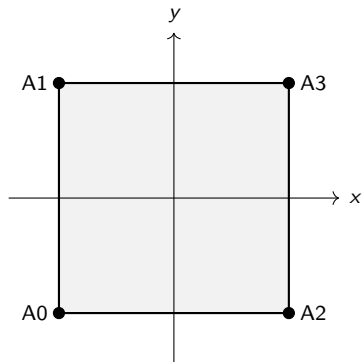
Exactly one threshold in the codebase is counted in *samples* rather than seconds:

MAX_CONSECUTIVE SATURATED = 30

at 348 Hz (the wire)	86 ms
at 23.5 Hz (4 coils)	~ 1.3 s

And it costs phase. $K_p = -0.040$ was seen on v0 driving *one* coil at 73 Hz. Today's four-channel config runs 3× slower, with correspondingly less margin at 1 Hz.

Four sensors, one rigid body



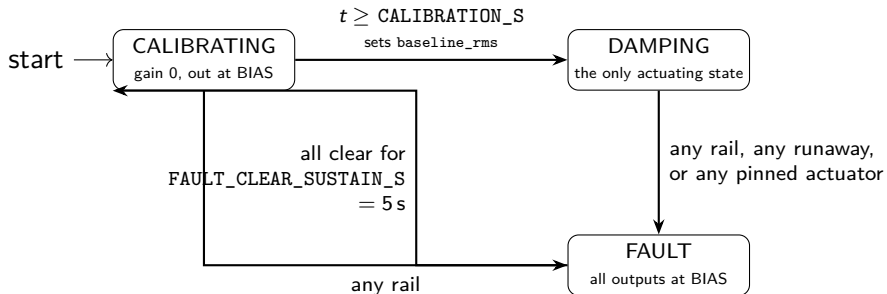
$$z_i = \text{LONG} + \text{PITCH} \cdot y_i + \text{YAW} \cdot x_i$$

mode	z_0	z_1	z_2	z_3
LONG	+	+	+	+
PITCH	-	+	-	+
YAW	-	-	+	+
BUTTERFLY	+	-	-	+

Four sensors, three rigid-body freedoms. BUTTERFLY is identically zero for a rigid body — any reading in it is one sensor disagreeing with the other three.

Consequences the whole project rests on: one OSEM/coil pair damps the *whole* mass (≈ 70 s); four together settle it in ≈ 17 s *provenance.md* §3, §4. That $4\times$ is why v1 exists, and it is why a quorum of *one* healthy channel is enough.

The supervisor: three states



No transition out of DAMPING except into FAULT, and no terminal state. Ctrl+C leaves through a finally block that returns every DAC channel to bias. CALIBRATION_S was 8 s on v0–v2 and is **20 s from v3 onward**.

v0 — the baseline

ENABLE_CHANNEL = [True, False, False, False]
P-only: $u = -K \cdot v$, KI_GAIN = KD_GAIN = 0.

What v0 already got right (and every version since inherits):

1. Runs continuously, not as fixed-duration sweep steps.
2. Measures its *own* baseline every (re)start, instead of borrowing one 45 s window from a 9-minute run.
3. A rail interlock on *raw counts*, separate from the RMS breaker — because a blind OSEM flatlines, which an RMS-only check reads as *perfect stability*.
4. Auto-recovery out of FAULT.
5. A lock detector that prints a real number.
6. Streaming CSV, flushed periodically, survives an unclean kill.
7. Gain scheduling with a slew-limited ramp.

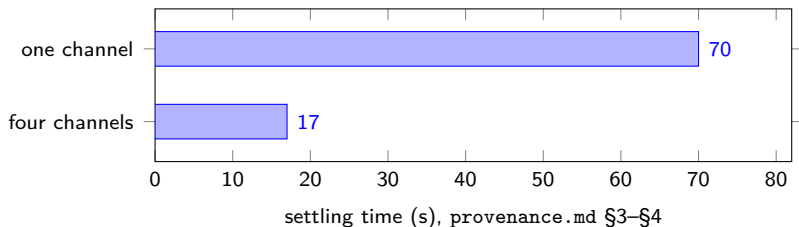
v0, measured through harness.py

ch0 amplitude ratio	0.188
time to lock	12.1 s
suite (when present)	18/18
undriven ch1/2/3	0.333 / 0.316 / 0.253

Three defects shipped with it, all present in v0, v1 and v2: saturation-latch, rail-threshold, runaway-baseline.

Plus a fourth nobody found until v9.

v1 — one line changed



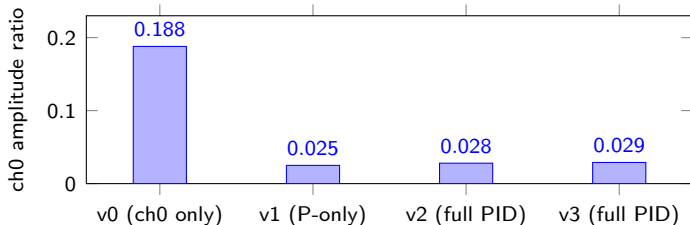
`ENABLE_CHANNEL = [True]*4`. Nothing else moved: same gains, same filters, same safety code, same three defects.

v0's docstring claim that only ch0 was validated was **already two days out of date** when v0 was frozen — §4 of the report documents all four damping simultaneously.

Why 4×, when it is still four SISO loops? The four loops act *in parallel* over the coupled modes, so the overall rate is closer to the *sum* of their contributions than to the single slowest coupling path.

Measured: ch0 ratio 0.188 → **0.025**, lock 12.1 → **10.7 s**.

v2 — I and D switched on, and the prediction that held



The I and D terms bought nothing in damping — exactly as predicted, because neither dissipates. And on hardware v2 was *worse*: its integrator was non-zero in **63%** of samples and pushed a pinned-actuator streak from **28 to 31** against a threshold of **30**.

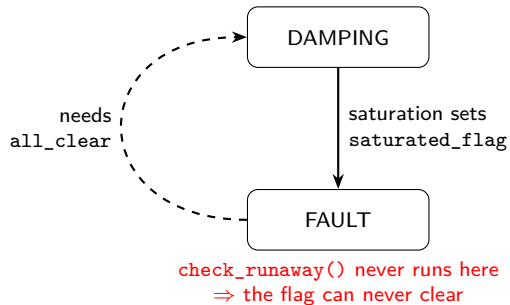
Peak $|I|$ was 0.084 V against a 0.150 V clamp — **I_CLAMP_V never even engaged**. Bounded is not the same as unwinding.

On the bench, 2026-08-03 — the only hardware evidence for v1–v3

	v1	v2	v3
Law	P-only	full PID	full PID
Lock time	13.8 s	13.9 s	14.5 s
Baseline (ch0)	0.6316 V	0.3158 V	0.1804 V
Peak bp	3.141 V	2.837 V	0.410 V
ADC clipped	2.19%	10.40%	0.00%
Samples with non-zero /	0 of 6939	4709 of 7513	all
Longest pinned run	28	31	0
Trip type	runaway	saturation	none
Outcome	recovered in 5.0 s	latched ~50 s	no fault

Do not compare ratio across these runs — it is normalised to each run's own baseline and those differ by 3.5×. Normalising the absolute locked RMS gives 0.139/0.128/0.115: indistinguishable. **These runs do not rank the three on damping quality.**

Defect 1: saturation-latch — reproduced on hardware



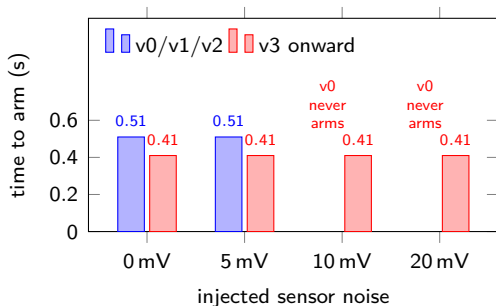
v0 recomputed `saturated_flag` only inside `check_runaway()`, which does not run in FAULT.

A saturation trip therefore pinned `all_clear` to **False forever** — permanent fault until a manual restart, flatly contradicting the auto-recovery the module advertises. A *runaway* trip recovered fine; **only saturation deadlocked**.

The fix (v3). Maintain the flag in `actuate()`, where the output is actually known — so it refreshes every sample in every state, including FAULT, where the output is held at bias and it clears on its own.

Reproduced on hardware, 2026-08-03. Across 16,966 FAULT samples v2 logged exactly **one distinct ratio value (3.6803)** — nothing was being recomputed at all.

Defect 2: rail-threshold — an interlock at the noise floor



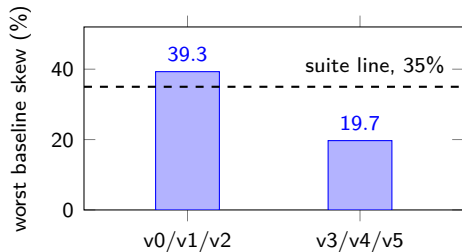
v0: RAIL_LOW_COUNTS = 3 of 1023 (~14.7 mV), and 250 *consecutive* samples under it. One noise sample above the line reset the count.

So above ~1 count RMS of dark noise the interlock silently **stopped arming** — and a blind channel then reports an amplitude ratio near zero, reading as the **steadiest axis on the rack** while the loop stays in DAMPING and declares itself locked.

v3: 12 counts, and **80% of a 0.5 s window** rather than an unbroken run — noise now *dilutes* the evidence instead of erasing it.

Also reproduced on hardware: v1 logged 246 genuinely clipped ADC samples (2.19%) and **the rail flag never set once**.

Defect 3: runaway-baseline — improved, *not* solved



One 6 V/s impulse swept across the calibration window
($t = 2/4/6$ s), rigid-body plant, **simulator only**.

v0: one RMS over a single 8 s window, then compared against for the rest of the run.

v3: calibrate 20 s, take the **median of 5 sub-windows**, so one bad sub-window is outvoted.

Read the *spread*, not any one column. On a retired seven-point sweep v0 came out *better* than v3 at $t = 5$ s and $t = 7$ s. A single fixed shock time can rank them backwards; what the median buys is a **bound**.

It does not help against sustained disturbance.

Gain is zero during calibration, so a shock rings down over a timescale comparable to the window itself and there is no clean sub-window left to prefer.

Labelled a partial fix in the code. A real fix needs a baseline that tracks, or a transient detector that restarts calibration — neither exists in any version.

v3 — the control law itself becomes a real PID

change	why
derivative on <i>measurement</i> , not on error	algebraically identical while the setpoint is a constant zero, but it cannot produce a derivative kick if that ever changes
back-calculation anti-windup , <code>TRACK_TC_S = 0.5 s</code>	the old scheme froze the accumulator at a hard limit; this bleeds it back at a defined time constant, so recovery from saturation is smooth rather than dependent on which side of a conditional the error fell on. <code>I_CLAMP_V</code> stays as a backstop
one <code>reset()</code> , on every entry to a non-actuating state	exactly one definition of bumpless re-entry

Two kinds of change kept deliberately separate — **the control law** above, **the three defect fixes** on the previous slides — so that a bench comparison can attribute an effect to one or the other.

`Channel.p_term / i_term / d_term / ki / kd` survive as *properties*, because the logger, the status line and the simulator's live gain editing all use those names.

v3 is one of only three controllers with `BENCH_STATUS = validated` (with v7 and v9). It is the minimal PID baseline to A/B against.

The v4 that failed: modal (MIMO) damping

The idea, and it is a good one. Four loops each fighting a plant the other three are also driving is the wrong decomposition. Control the *motion*, not the sensors:

1. Accumulate the 4×4 covariance during calibration — its eigenvectors *are* the observed modes.
2. A new SYSID state: dither each coil in turn, measure how the modes answer $\Rightarrow$ the actuation matrix B .
3. Damp per *mode*; distribute through B^+ .

Its own selling point: it would *discover* that ch2 pushes the other way instead of having to be told.

It did not. Written and deleted the same day, 2026-08-03.

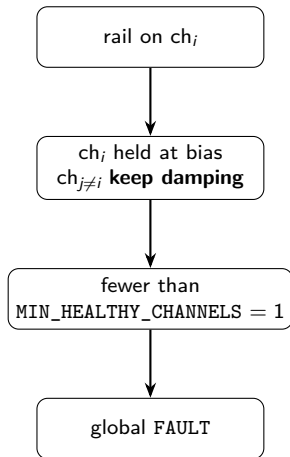
ground truth	$[-0.59, -0.45, +2.45, -0.51]$
vel $\times$ PRBS	$[+2.0, +0.2, -0.53, -1.27]$
acc $\times$ PRBS	$[-0.71, -0.47, -0.86, -1.96]$

Neither recovered ch2's sign. Two causes, both diagnosed: zero-lag correlation on *velocity* has no coherent term, because force acts on *acceleration*; and a ~ 40 -bit PRBS gives $\sim 16\%$ inter-channel crosstalk — the size of the differences being measured.

Its *mode estimator* was fine: 98.8% / 1.2% mode content.

The hand-measured constant beat the automatic method.

v4 — auto-disable: demote the axis, do not freeze the rig



The problem. v3 reads `any_rail = any(ch.rail_fault ...)` and freezes all four. One OSEM losing its flag stops *all* damping — **at exactly the moment of largest excursion**, which is what made the flag leave range.

A quorum of one is not a guess: provenance.md §3 measured one OSEM/coil pair damping the whole rigid body.

condition	v4	why
rail	demote	local <i>sensing</i> failure: that measurement is garbage
runaway	global	not localisable — four loops, one body
saturation	global	measurement fine, only the output clipped

Saturation is deliberately *not* demoted: a saturated channel is still pushing the right way at full authority.

v5 — four channels become four lanes

	total	code	logic	main()
v3	908	515	400	64
v4	1172	577	460	64
v5	533	356	270	48

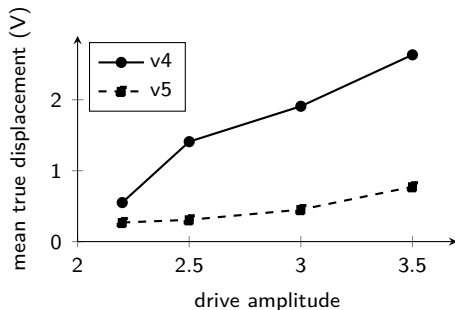
Channel went from **231 lines to 13**: it stopped being where state lives and became a *view* onto the controller's arrays, so `sim/server.py` and `harness.py` kept working unchanged.

The evidence it is the same controller — every printed observable matches v4 *exactly*, on two different plants:

observable	v4, and v5 identically
ratios	0.029 / 0.098 / 0.045 / 0.132
lock	10.6 s
peak $P/I/D$	0.0881 / 0.0644 / 0.0412 V
re-arm	2.10 s

The one thing that got worse: speed, and in the direction you would not guess — **57.5 μ s/step against v4's 42.4**. numpy's per-call overhead on a length-4 array beats four scalar operations. Both are $\sim 50\times$ inside the 2.88 ms budget, so it is irrelevant on hardware.

v5 — fast-refault: stop re-measuring during the event



Simulator *ground truth* over 300 s — not the controller's own estimate.

The problem. v0–v4 answer every fault the same way: wait for all-clear, then re-calibrate. That is CALIBRATION_S of open loop at zero gain — correct for a transient, *wrong* for a disturbance that has not gone away. The optic rings straight back up to the amplitude that faulted it.

200 s @ drive 2.5	% DAMPING	faults	locks
v0 / v1 / v2	0.6%	1	never
v3 / v4	4.1%	7	never
v5	87.5%	1	10.2 s

The fix is small. If the previous engagement lasted under FAST_REFAULT_S = 15 s, the cause is evidently still present — re-engage on the baseline already in hand. Bounded by MAX_BASELINE_REUSE = 4.

Found by stressing the simulator across clipping regimes, not by reading the code.

v5.5 — eight OSEMs, and why it never locked

v5 with $N = 8$ and the full map `DAC_CHANNELS = [1,3,5,7,0,2,4,6]`. **Not a new rung**: same control law, constants, FIXES and state machine.

Going $4 \rightarrow 8$ was a config change rather than a rewrite *only because v5 keeps channel state in arrays*.

All eight sit on one rigid body, so every semantic held unchanged: quorum of one, one global FAULT, one lock claim.

Channels 4–7 shipped disabled — their sign is unknown, and a wrong-signed channel *pumps*. ch2 was *discovered* from bench data, not predicted.

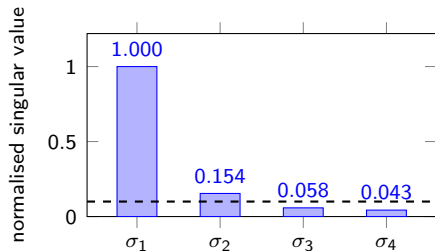
The simulator cannot test this file.

`sim/server.py` models one rigid body with exactly four OSEMs — four corners and a 4×4 projection. There is no measured geometry or coil gain for 4–7, and **inventing them would mean validating against fabricated physics**.
make check skips it and says why.

It has never locked. On the bench, ch0–ch3 damped at ratio 0.07–0.26 while ch4–ch7 — calibrated at $\sim 0.002\text{ V}$ — **faulted the whole rig six times in 120 s**.

An OSEM carrying no *in-band* signal does not rail — and that, not disconnection, is what a4/a6/a7 turn out to be. It sits mid-scale with a flat bandpassed output, so auto-disable never fires; it calibrates a near-zero baseline, and then any noise reads as a runaway. **A minimum-plausible-baseline check is the missing guard.**

v6 — measuring the actuation matrix properly



Dashed: the 10% retention line.

Directions above it: 2; expected: 2 — bench

2026-08-04

KIND = "sysid" — **not a controller**. It damps nothing and closes no loop; the harness refuses to simulate it.

Stepped sine, one coil at a time. While coil j drives, no other coil is. Nothing to disentangle, no matrix to invert, no assumption about quadrature — **magnitude and phase both fall out of the lock-in**. That is precisely what the modal v4 got wrong.

Four coils drive two DOF, confirmed. A rank that is not 2 means the *measurement* is wrong rather than the optic.

But only 4 of 12 frequencies were usable — ch2 railed in 17 of 48 steps — and two of those four are near-identical across coils, i.e. ambient-dominated.
A valid measurement on a quarter of the sweep, not yet a matrix to build a modal controller on.

The DC actuation matrix — and the conclusion it will not support

coils 0–3					coils 4–7			
a0	−105	−107	−48	+19	−29	−21	−28	−26
a1	−49	−68	−33	+18	+5	+7	+4	+8
a2	+35	+66	+213	−141	−25	−22	−13	−26
a3	+36	+14	+58	−51	−1	−2	+2	−1
a4	+0	+2	+2	+1	+8	+8	+2	+1
a5	+4	+0	+0	−6	−0	−7	−1	+8
a6	+1	+4	+2	+3	+3	+3	+4	+6
a7	−0	+3	+1	+0	+1	−1	+6	+7

$d(\text{counts}_i)/d(\text{bias}_j)$, one coil stepped at a time, bench

2026-08-04.

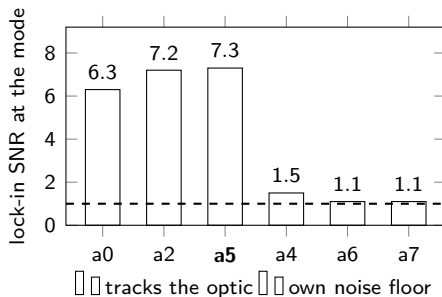
Block means of response		
	coils 0–3	coils 4–7
sensors 0–3	66	14
sensors 4–7	2	4

Coils 4–7 work — they move a0–a3 by ~ 25 counts/V. **Sensors a4–a7 do not respond to anything**, ≤ 8 counts/V.

We read that as “a4–a7 are not wired”. It was wrong — and the next slide is why. A DC slope against a fixed threshold cannot separate *no sensor* from *a working sensor with small gain*.

Note a3: the common-mode sweep read +43 counts/V, but coil3 $\rightarrow$ a3 alone is −51. **Common-mode slopes are not per-coil slopes.**

Measured today: all eight *are* wired — in-band vs out-of-band



Lock-in on the two long 8-channel logs (118s and 148s) at the rigid-body mode frequencies. Dashed: SNR = 1, a channel's own noise floor.

`analysis/blind_check.py`, `analysis/a5_check.py`

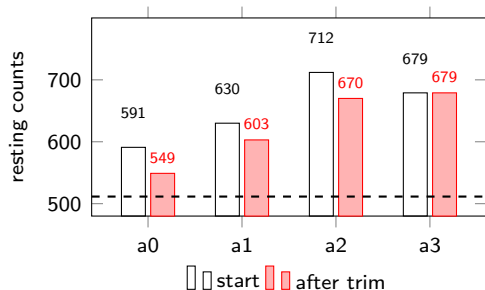
All eight OSEMs are electrically connected — confirmed on a scope, 2026-08-06. The split is not wired/unwired, it is *in band/out of band*:

power in 0.4–3 Hz		
a0–a3	77–99%	modes at 1.01 and 1.66 Hz
a5	22–71%	tracks 1.01 Hz, at $\sim 1/12$ of a0's counts/m
a4, a6, a7	0.1–9%	a narrow 6.19 Hz line (12.4 Hz harmonic on a7)

It is not an Arduino or sampling problem. a3 is read *immediately before* a4 and carries a large clean 1.66 Hz signal, yet $r(a4, a3) = -0.029 / +0.004$: no mux leakage from the strongest possible source. And a5, read straight *after* a4, is clean. The signal is not lost between sensor and stream.

The “not wired” call came from a **DC-response threshold** and a **raw time-domain correlation**. Both are blind to a small coherent signal; the spectral test found it in one pass. **Wrong statistic, not wrong**

v7 — bias-trim: move where the optic hangs



Dashed: ADC mid-scale, 511.5 counts. Bench 2026-08-04. a3 was already at its best step.

Coil bias is a **DC force**, so stepping it moves where the optic hangs. One `BIAS_QUANTUM` = 0.25 V at a time, kept only if the **total** offset across all channels improved.

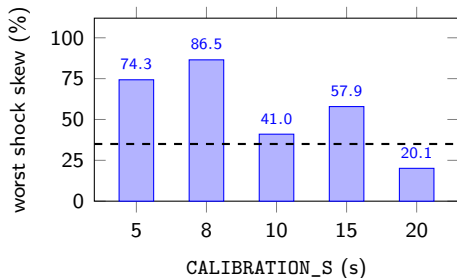
bench	2026-08-04	start	end
v7 (trim)		565	455 counts
v5 (control)		566	546 (drift)

Lock times 11.2 / 11.0 / 12.4 s — **the trim costs nothing.**

Two steps were rejected and frozen, both by the *total* test: ch1 → 0.50 V (486 → 528) and ch3 → 1.25 V (538 → 561). Each helped its own channel and *hurt the rig* — the four-coils-two-DOF coupling v6 measured.

It cannot finish the job: centring wants $-1.93 \dots + 3.77$ V against a 2.5 V DAC. The rest is mechanical or TIA offset.

v8 — stop paying 20 s of zero gain for a baseline



Dashed: the 35% assertion line. **Why not simply a shorter window** — research.md item 6. Every shorter window fails, some worse than the un-fixed defect (39.3%).

The cost. Bench, four channels: 20 s calibration + ~11 s to lock = **65% of wall clock at zero gain**, plus 25 s of open loop after every fault.

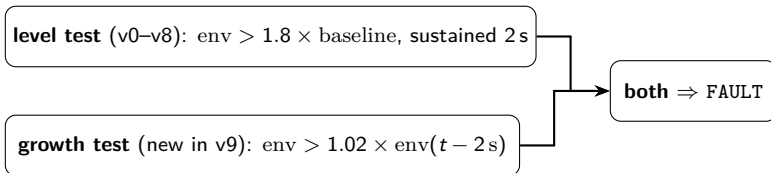
1. fast-calib. CALIBRATION_S becomes a *ceiling*. Sub-windows accumulate; calibration ends as soon as the trailing ones agree the floor has stopped moving. Simulator: 20.00 → **6.01 s**, time-to-lock 30.6 → **16.5 s**, and shock skew *improves* 19.7 → 17.6%.

2. warm-restart. A fault after a *healthy* engagement re-engages on the baseline in hand. Two 15 V/s kicks: v4/v5 never re-lock inside 115 s; v8 re-engages 5 s after all-clear and re-locks 10.7 s later.

On the bench fast-calib never fired.

CALIB_AGREE_TOL = 1.20 was tuned on simulator noise, one RNG seed. It degrades safely to v5's estimator — the designed fallback — but the saving is **unconfirmed on hardware**.

v9 — the runaway breaker was asking the wrong question



A **runaway** is the loop pumping energy *in* — which means the envelope **growing**.

A **ringdown** is a large but **falling** envelope — which means the loop *succeeding*.

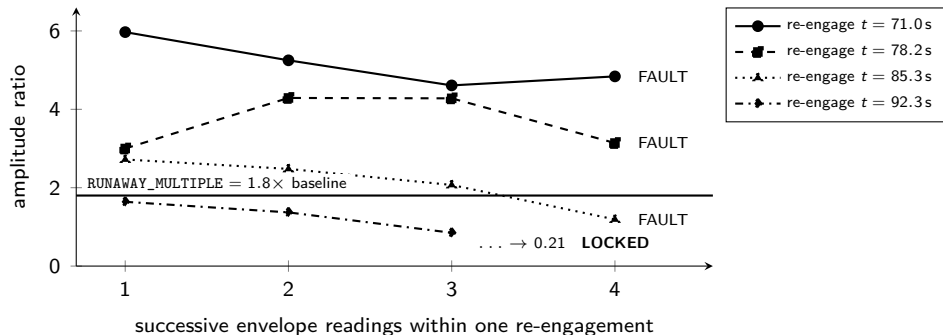
A **level test cannot tell them apart**. So after a kick it trips on a timer, every `RUNAWAY_SUSTAIN_S`, all the way down the ringdown.

This defect was in every version back to v0. v8 merely re-engaged fast enough to hit it repeatedly, which is how it was found.

`RUNAWAY_GROWTH_FRAC = 1.02` gives 2% of headroom so envelope noise alone cannot read as growth. `RUNAWAY_TREND_LAG_S = 2.0` is one `ENVELOPE_WINDOW_S`, so the two estimates barely overlap.

The suite confirms it *still* trips on a genuinely pumped resonance — the growth gate rejects ringdowns without letting a real runaway through.

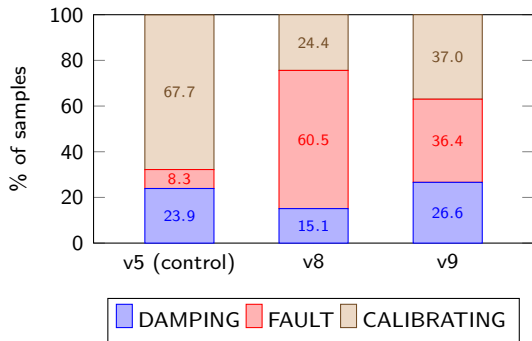
v9 — the bench evidence: the envelope *falling* through every fault



Measured on **v8**, hardware, 240 s, three kicks: **ten faults**, and the envelope was *falling* through them.

Only the re-engagement that happened to start at 1.64 — just under the line — survived.

v9 — what the growth gate bought



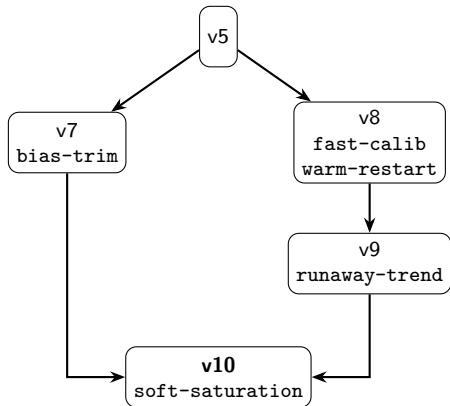
same 3-kick schedule	faults	locks
v5 (control)	1	2
v8	10	3
v9	4	4

v9 re-locked after every kick —
 $t = 31.3/83.8/151.3/205.3$ s.

Most DAMPING, most locks, and less than half the faults. v5 looks calm only because it spends 68% of the run calibrating.

v9's remaining 36% in FAULT is
 $\text{FAULT_CLEAR_SUSTAIN_S} = 5\text{ s} \times 4$
recoveries plus the gain ramp — **no longer thrash**. Gating re-engagement on amplitude rather than a fixed 5 s would cut it further.

v10 — merging the branches, and soft-saturation



The new fix. v9 faulted the whole rig after 30 consecutive samples pinned against a rail. But **clipping removes authority in one direction only** — an output pinned at v_{max} still pulls *down* at full strength. A clipped loop is a **weakened** loop, not a broken one, and it is usually the thing bringing the optic back.

$$\text{sat_trip} = (\text{sat_streak} > 30) \wedge \neg(\text{env} < 0.98 \text{env}(t-2s))$$

Same level-vs-trend correction runaway-trend made — one interlock over. Same `env_hist` deque, same lag, so “falling” means one thing to both.

Why the merge needed a third change. A bias step *is* a force step: it rings the pendulum at ~ 1 Hz, inside the band both trend tests read. So a step **invalidates** the history rather than suspending the interlocks — a 16s hole every `TRIM_PERIOD_S` would be open nearly always.

v10 — status, honestly

BENCH STATUS

```
simulator suite, harness.py --test v10
```

whole tree, make check (v3 23, v7 35, v9 36, v10 37)

untested

37 passed, 0 failed

131 passed, 0 failed

A rejected gate, recorded because it looks right and is not. The first draft required *clip* excess — “still demanding more than the rail can give.”

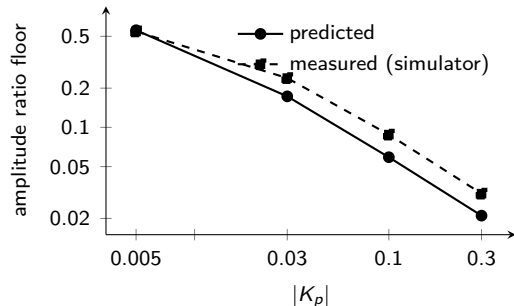
But **back-calculation anti-windup exists precisely to drive that residual toward zero** under sustained clipping. The test went false exactly when saturation was worst, and the suite caught it.

An anti-windup residual cannot measure unmet demand.

What is honest to say today:

- ▶ bias-trim's numbers are v7's bench result, not v10's.
- ▶ soft-saturation has only ever run in the simulator — which models clipping, but **not the coil driver behind it.**
- ▶ The old [FAIL] is gone because the *assertion* was re-expressed, not because behaviour changed: over-gain now has to keep damping while clipped, and it does — v9 faults once and peaks at 1.527; v10 never faults and peaks at 0.853.
- ▶ Only **v3, v7 and v9** read validated.

The floor: why more gain is the only way down



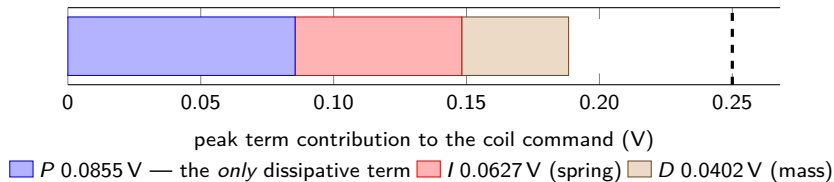
The ratio settles at **0.23** — and that is the *steady state of a continuously forced oscillator*, not a failure to converge. Remove the drive mid-run and it decays with $\tau = 3.4\text{ s}$ damped against 15.9 s gain-off (predicted $2.76 / 15.92$).

The floor tracks $\frac{\text{intrinsic}}{\text{intrinsic} + \text{added}}$ damping.

The floor falls with gain; gain is capped by the -0.04 question. Those two facts are the whole reason `research.md` item 3 is ranked where it is.

The residual 0.0089 V is the actuator's 0.5 mV deadband and 10 ms throttle, not sensing — $0.0089/0.0087/0.0094\text{ V}$ at $20/5/0\text{ mV}$ of injected sensor noise.

Measured today: 40% of the actuator goes to terms that remove no energy



Dashed: the actuator half-window, $V_{MAX} - \text{BIAS} = 0.25 \text{ V}$.

I is **73%** of P . D is **47%** of P . The three sum to **0.188 V** of a $\pm 0.25 \text{ V}$ window.

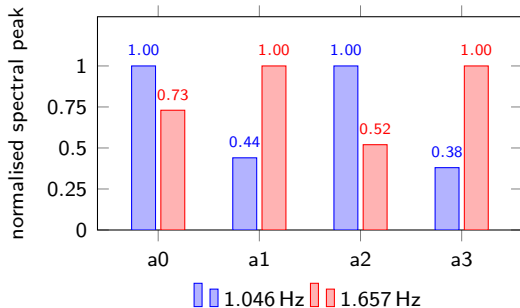
$I + D$ alone is **0.103 V** — **~40% of the actuator authority**, in a rig whose central problem is headroom, spent on the two terms that by construction remove *no* energy.

And v2 already measured what they buy: **nothing**. 0.028 against v1's P-only 0.025.

provenance.md contains no justification for KI_GAIN or KD_GAIN at all — not a scope trace, not a sweep, not a sentence. -0.030 has one. These two have none.

Zeroing them is the cheapest headroom in the repo, and v3 is already the A/B baseline for it.

Measured today: two modes, out of the sensor spectra alone



Welch spectra of the raw counts, CALIBRATING windows,
27 bench logs. `analysis/out/modes.csv`

a0 and a2 peak at 1.046 Hz. a1 and a3 peak at 1.657 Hz. Two frequencies, four sensors, a clean split — and in the corner layout that is the lower pair against the upper pair.

This is the same result v6 got, by a completely different route. v6 drove each coil and counted singular values above 10%: two. This drove *nothing* — it is the open-loop sensor spectra, gain at zero.

Four coils, two degrees of freedom — now confirmed from both the actuation side and the sensing side.

Both modes sit inside the 0.4–3 Hz bandpass, so the loop does see them. But it damps them as four independent SISO axes, which is exactly the decomposition the modal v4 argued was wrong.

What the simulator can and cannot tell you

It runs the *real* controller — every four-channel version is imported and stepped, not reimplemented. Only `serial` and `DACController` are stubbed.

The plant is a linear second-order oscillator per mode with **exactly one nonlinearity**: the ADC, which offsets each channel to its measured resting count, rounds to an integer, and hard-clips at 0/1023. That is why clipping emerges on its own — and with the bench's asymmetry.

What it cannot tell you:

- ▶ **A safe gain.** More negative K_p is monotonically more damping all the way to -0.6 . The ADC model reproduces *clipping*, not instability.
- ▶ **A wrong coil map.** `sim/server.py` round-trips its held-voltage dict through whatever map the controller declares. **Only the bench catches one.**

And `COIL_GAIN` = [1.00, 0.76, -4.15 , 0.86] is *inferred* from Table 1, not measured — the report characterises no actuator. **One significant figure is the honest precision.** Q and drive amplitude are sliders because neither was ever characterised.

Still open

1. $K_p = -0.040$ **has never been applied on hardware.** Across **21** logged runs the largest $|\text{gain}|$ ever commanded is **0.035** — CAPTURE_GAIN, on ch0/1/3 — and **0.012** on ch2. Nothing has ever been driven at the limit that caps the whole project. And the one run that saw -0.040 was v0 driving *one* coil at 73 Hz; we now run four at 23.5. *analysis/kp040.md*
2. **Zero KI_GAIN and KD_GAIN and re-measure.** 40% of the actuator, no justification in provenance.md, and v2 already showed no damping benefit. Free headroom, no hardware change, v3 is the A/B baseline.
3. **Chase the 6.19 Hz line; bring a5 into the loop.** All eight OSEMs *are* wired. a4/a6/a7 put 0.1–9% of their power in band and are dominated by that one narrow line, while **a5 already tracks the optic** at $\sim 1/12$ a0's sensitivity. Everything eight-channel waits here — **an interference and alignment job, not a wiring job and not a code job.**
4. **Closed-loop dither sysid.** Both attempts were limited by the same thing: open loop, nothing damping, so the optic rings up and ch2 clips. sysid.run() already divides by the *actually commanded* u , which is most of what that needs.
5. **Bench-tune CALIB_AGREE_TOL** from the logs already recorded, so fast-calib fires on hardware.
6. **Run v10 on the bench.**
7. **A minimum-plausible-baseline guard** — an OSEM with no in-band signal does not rail, so no version handles it.

The ladder, end to end

	channels	I/D	what it added	ch0 ratio	lock	bench
v0	ch0 only	zeroed	<i>the baseline</i>	0.188	12.1 s	2026-07-15 (scope)
v1	all four	zeroed	all four channels on	0.025	10.7 s	2026-08-03
v2	all four	live	real K_i , K_d	0.028	10.5 s	2026-08-03
v3	all four	live	all 3 defects fixed	0.029	10.6 s	validated
<i>v4 (modal) MIMO on a learned basis — written and deleted 2026-08-03</i>						
v4	all four	live	auto-disable	0.029	10.6 s	2026-08-04
v5	all four	live	fast-refault	0.029	10.6 s	2026-08-04
v5.5	<i>all eight</i>	live	eight-OSEM build	—	<i>never locked</i>	a4/a6/a7 out of band
v6	<i>bench tool</i>	— stepped-sine actuation matrix, closes no loop				2026-08-04
v7	all four	live	bias-trim	0.029	10.6 s	validated
v8	all four	live	fast-calib, warm-restart	0.029	10.5 s	2026-08-04
v9	all four	live	runaway-trend	0.029	10.5 s	validated
v10	all four	live	soft-saturation + merge	—	—	untested

Ratios and lock times are **simulator** numbers through `harness.py` on the rigid-body plant. **The control law has not changed since v2** — everything from v3 on is the supervisor around it. Pruned **2026-08-06** to **v3, v6, v7, v9, v10**; v0–v2, both v4s, v5, v5.5, v8 and the v6.1/v6.5/v6.6 sysid variants are in git history, and their numbers stand because they were measured while the files were present. `make check` today: **131 passed, 0 failed**;

`analysis/out/*.csv` re-analyses all **27** bench logs.